VQF 姿态滤波算法详解

原理推导 · 逐行代码解析 · 与 EKF 的对比

源码：Library/Src/VQF_filter.c（367 行）、Library/Inc/VQF_filter.h（86 行）

代码字体：Maple Mono CF CN（与 VSCode 一致）

一、姿态估计要解决什么问题

任何姿态估计器都在做同一件事：**用两个都不完美的传感器，拼出一个更好的结果。**

1.1 两个传感器的互补缺陷

	陀螺仪	加速度计
测量对象	角速度（rad/s）	比力（m/s^2），静止时≈重力反方向
短期特性	噪声小、响应快，**短时非常准**	噪声大、易受振动干扰
长期特性	积分产生零偏漂移，**越积越歪**	长期稳定，没有累积误差
致命弱点	零偏与温漂导致角度缓慢漂走	**无法区分重力与线性加速度**
可观测的量	三轴旋转	**只能观测倾斜，完全无法观测偏航**

关键矛盾：陀螺仪短期准但长期漂，加速度计长期准但短期差。所以融合的思路天然就是——**用加速度计的长稳特性，去修正陀螺仪的长漂特性。**

1.2 为什么这个「修正」不能随便做

朴素做法是「加速度算出的倾角」与「陀螺积分出的倾角」做加权平均（互补滤波）。这能跑，但有几个隐患：

- 加速度计在**机动时测量的是「重力 + 运动加速度」**，此时算出的倾角是错的。直接加权平均会把运动误差混进姿态。
- 如果观测矩阵或加权方式设计不严，加速度计的噪声会**漏进偏航角**，造成航向缓慢乱转。
- 极端姿态（如接近 180° 翻转）下公式出现奇异，可能输出 NaN。

VQF 的设计目标，就是**用确定性的几何运算，把上述问题一次性解决干净**，同时保持极低的计算量。

二、VQF 的核心思想：双四元数解耦

2.1 一句话概括

VQF 把姿态拆成两个四元数，各自只负责一件事：

```
q = accQuat ⊗ gyrQuat
    │   │
    │   └── gyrQuat: 纯陀螺积分，负责「高频、快速」的姿态变化
    └────── accQuat: 纯重力校正，负责「低频、缓慢」的倾斜基准
```

这个分解是 VQF 全部巧妙之处。它的好处是：**两条通路各自独立演进，互不污染。**

2.2 gyrQuat —— 只信陀螺

```
vqf_real_t gyrNoBias[3] = {gyr[0]-f->bias[0], gyr[1]-f->bias[1], gyr[2]-f->bias[2]};
vqf_real_t gyrNorm = vqf_norm(gyrNoBias, 3);
vqf_real_t angle = gyrNorm * f->gyrTs;
if (gyrNorm > EPS) {
```

```
vqf_real_t c = cosf(angle/2);
vqf_real_t s = sinf(angle/2)/gyrNorm;
vqf_real_t gyrStepQuat[4] = {c, s*gyrNoBias[0], s*gyrNoBias[1], s*gyrNoBias[2]};
vqf_quatMultiply(f->gyrQuat, gyrStepQuat, f->gyrQuat);
vqf_normalize(f->gyrQuat, 4);
}
```

这是标准的**轴角增量积分**。设本步角速度为 ω、时间 dt，则转过角度 θ = |ω|·dt，旋转轴为 ω/|ω|。对应的增量四元数是 [cos(θ/2), sin(θ/2)·ω]。把它左乘到 gyrQuat 上，就完成了这一步的姿态推进。

注意这里用的是**扣除零偏后**的角速度。零偏由后面的卡尔曼估计器给出——这是陀螺长漂被抑制的根源。

2.3 accQuat —— 只信重力，且只修倾斜

这是 VQF 最关键的一段。逐步看：

```
// ① 用陀螺四元数把机体加速度转到「地理系」
vqf_quatRotate(f->gyrQuat, acc, accEarth);

// ② 低通滤波，压掉线性加速度与振动
filterVec(accEarth, 3, f->tauAcc, f->accTs, f->accLpB, f->accLpA,
f->accLpState, f->lastAccLp);

// ③ 再用 accQuat 修正，得到「当前估计的重力方向」
vqf_quatRotate(f->accQuat, f->lastAccLp, accEarth);
vqf_normalize(accEarth, 3);

// ④ 构造把该向量旋到 +z 的最短旋转四元数
vqf_real_t q_w = sqrtf((accEarth[2]+1)/2);
accCorrQuat[0] = q_w;
accCorrQuat[1] = 0.5f*accEarth[1]/q_w;
accCorrQuat[2] = -0.5f*accEarth[0]/q_w;
accCorrQuat[3] = 0; // ← 注意这一行

// ⑤ 累积到 accQuat
vqf_quatMultiply(accCorrQuat, f->accQuat, f->accQuat);
vqf_normalize(f->accQuat, 4);
```

2.4 关键洞察：第 ④ 步的 z 分量为什么是 0

这是理解 VQF 的钥匙。设需要旋转的向量为 v = (v_x, v_y, v_z)，目标是把它转到 +z 轴。把 v 转到 +z 的**最短旋转**四元数为：

```
q ∝ [ 1 + v·ẑ , v × ẑ ]
= [ 1 + v_z , (v_y·1 − v_z·0), (v_z·0 − v_x·1), 0 ]
= [ 1 + v_z , v_y , −v_x , 0 ]

归一化：|q|² = (1+v_z)² + v_y² + v_x²
= (1+v_z)² + (1 − v_z²) ← 因 |v| = 1
= 2(1 + v_z)

于是 q_w = (1+v_z)/√(2(1+v_z)) = √((1+v_z)/2) ← 对应代码 268 行
q_x = v_y / √(2(1+v_z)) = 0.5·v_y / q_w ← 对应代码 270 行
```

```
q_y = -v_x / √(2(1+v_z)) = -0.5·v_x / q_w ← 对应代码 271 行
q_z = 0 ← 对应代码 271 行
```

$\mathbf{v} \times \hat{z}$ 的 z 分量恒为 0，因为两个向量的叉积垂直于它们张成的平面，而 $\hat{z}$ 就在这个平面内。

这个数学事实带来了工程上极其重要的性质：

- **旋转轴落在水平面内**（四元数虚部的 z 分量为 0，说明转轴 $\perp \hat{z}$）。
- 绕水平轴的旋转**只改变倾斜，不改变偏航。**
- 因此**加速度计永远不会污染航向角**——这是 VQF 与很多互补滤波/EKF 实现的分水岭。很多实现里「静止时 yaw 慢慢乱转」的毛病，根源就是重力观测漏进了偏航通道。

同时也要理解它的代价：**偏航角完全由陀螺积分决定。**没有磁力计时，偏航的长期精度只能靠陀螺零偏估计来维持——这正是 VQF 要花大力气做零偏卡尔曼的原因。

2.5 为什么低通滤波放在「地理系」

代码 ② 步是对 accEarth（已转到地理系的加速度）做低通，而不是对原始机体加速度做。这个顺序很重要：

- 在地理系里，**重力方向是恒定不变的**（始终是 +z）。
- 而机体的旋转已经被 gyrQuat 剔除掉了。
- 所以低通滤波器面对的「有用信号」是一个近乎恒定的向量，而「干扰信号」是运动加速度——**两者在频域上分离得非常干净。**
- 如果在机体坐标系滤波，重力分量会随姿态高速旋转，低通会把重力本身也当成高频信号抹掉，导致姿态失效。

三、二阶巴特沃斯低通滤波器的设计

3.1 为什么用二阶而不是一阶

一阶 IIR 的滚降只有 −20 dB/十倍频，对振动和运动加速度的抑制不够。二阶巴特沃斯有 −40 dB/十倍频，且在通带内**最平坦**（无峰值、无纹波），非常适合「既要平滑，又不能引入谐振」的姿态场景。

3.2 双线性变换（Tustin）

模拟滤波器无法直接跑在数字芯片上，需要用双线性变换映射到 z 域。VQF 采用的是标准的预畸变形式（代码 61~69 行）：

```
fc = (√2 / (2π)) / tau // 时间常数 → 截止频率
C = tan(π · fc · Ts) // 预畸变
D = C² + √2·C + 1

b0 = C² / D // 分子：对称的 [b0, 2b0, b0]
b1 = 2·b0
b2 = b0

a1 = 2(C² - 1) / D // 分母
a2 = (1 - √2·C + C²) / D
```

其中 √2 来自巴特沃斯的品质因数 Q = 1/√2。常数组合 (1, √2, 1) 正是二阶巴特沃斯分母多项式的标准形式。

关于 fc 的系数 √2/(2π)：常见的一阶滤波器用 fc = 1/(2π·tau)。VQF 用 √2 倍的更高频率，目的是让二阶滤波器的**时间响应**与「同样 tau 的一阶滤波器」等效（阶数越高，达到同样稳定时间需要更宽带宽）。这是工程约定，调参时应把它当作「时间常数」而不是「−3 dB 频率」来理解。

3.3 直接II型转置（DF2T）结构

```
y = b0·x + s0
s0 = b1·x − a1·y + s1
s1 = b2·x − a2·y
```

DF2T 的优点：**状态数最少**（N 阶只需 N 个状态）、**数值特性好**（相比直接I型，中间变量更不容易溢出）。这是嵌入式 IIR 的标准实现方式。

3.4 一个精巧的启动技巧：用 NaN 当哨兵

IIR 滤波器有个经典麻烦：**初值怎么给？**如果状态清零，滤波器会从 0 缓慢爬升到真实值，这段时间输出的加速度是错的，姿态会被带偏好几秒。

VQF 的解法（代码 83~103 行）非常巧妙：

- 初始化时把滤波器状态全部置为 **NaN**。
- 每步检查 isnan(state[0])：如果是 NaN，说明还处于启动阶段。
- 启动阶段**不用 IIR，改用算术平均**，把已积累样本求均值作为输出。这样从第一个样本开始输出就是合理的。
- 同时借用 state[1] 当计数器、state[2..] 当累加器。
- 当累计时长达到一个时间常数 tau，就用当前均值**反算 IIR 的稳态初值**，然后切换到正常的 IIR 递推。

好处是：**零额外内存、零额外分支标志**——用一个 NaN 就同时表达了「未初始化」和「已初始化」两种状态。这种手法在资源受限的 MCU 代码里很常见。

反算初值的公式（代码 71~74 行）：要让「输入恒为 x0」时输出直接等于 x0，由 DF2T 稳态方程可得

```
s0 = x0·(1 − b0)
s1 = x0·(b2 − a2)
```

四、静止检测

静止检测决定了「什么时候可以用最可信的方式估计零偏」。VQF 同时用陀螺和加速度两路判据，任意一路显示「在动」就否决静止。

判据	做法	阈值	能识别什么
陀螺波动	原始值 − 低通值，取平方和	restThGyr = 2.0 °/s	角速度是否稳定；转动时低通跟不上，差值立刻变大
陀螺幅度	低通输出的绝对值	biasClip = 2.0 °/s	平滑后是否仍在明显转动
加速度波动	原始值 − 低通值，取平方和	restThAcc = 0.5 m/s²	区分「静止」与「匀速直线运动」

判据	做法	阈值	能识别什么
持续时间	连续满足以上条件累计 restT	restMinT = 1.5 s	排除瞬时停顿造成的误判

为什么要 1.5 秒的下限（restMinT）？如果只要一瞬满足就认为静止，那么机器人每次减速停顿都会被当成静止，而此时陀螺输出里还残留着真实角速度——会被当作零偏吃进去，**反而把零偏估计带坏。**1.5 秒是一个经验上安全的时长。

加速度判据的作用尤其容易被忽视：**匀速直线运动时，加速度计的输出模值仍是 g 且方向不变**，仅凭陀螺判据会误判为静止。两路合起来才可靠。

五、陀螺零偏的卡尔曼估计

5.1 为什么要专门做零偏估计

没有磁力计时，**偏航角唯一的信息来源就是陀螺积分。**陀螺零偏哪怕只有 0.1 °/s，一分钟就会积累 6° 的航向误差，十分钟就是 60°，直接报废。所以零偏估计不是锦上添花，而是必需品。

VQF 用一个 **3 维卡尔曼滤波器**在线估计零偏（状态就是 bias[3]），但它的测量模型和常规 EKF 很不一样：它**借用姿态通道的残差**作为观测。

5.2 两路观测

	静止观测	运动观测
触发条件	restDetected == true	restDetected == false
观测方程	e = ω_lp − bias	e = −g_y/Ts + (R·bias)_x − (R·bias)_x^pred …
物理含义	静止时真实角速度为 0，所以平滑后的陀螺读数本身就是零偏	姿态与重力方向的失配速率，正比于零偏在水平面的投影
权重	biasRestW（最大）	biasMotionW（水平）、biasVerticalW（竖直）
精度	**最高**	较低，但能在持续运动中维持零偏不发散

5.3 竖直方向的可观测性问题

这是 VQF 里最需要理解的一个设计。考虑一个物理事实：

绕重力轴（偏航轴）的陀螺零偏，靠重力观测永远无法区分。因为无论偏航角是多少，重力向量在机体坐标系里的投影都不变。换句话说，偏航零偏在重力观测下是**不可观测的。**

如果在卡尔曼里一视同仁地更新三个轴，那么观测噪声会通过数值耦合**漏进不可观测的那个方向**，导致偏航零偏估计随机游走，甚至发散。

VQF 的处理很直接（代码 176 行）：

```
f->biasVerticalW = f->biasMotionW / max(f->biasVerticalForgettingFactor, 1e-10f);
// ↑ 默认 0.0001
```

```
// 结果：biasVerticalW 被放大到 biasMotionW 的 10000 倍
```

在卡尔曼公式 $K = P \cdot R^T \cdot (R \cdot P \cdot R^T + W)^{-1}$ 中，W 出现在分母。W 越大 → 增益 K 越小 → 该方向的更新量趋近于 0，**等价于「放弃估计这个方向」**。

这就是一种朴素但有效的**可观测性处理**：不去做复杂的可观测性分析（如 SVD 分解、零空间投影），而是用一个大权重把不可观测方向「压死」。**在 EKF 里，这类问题必须通过协方差矩阵的精细调节或专门的投影算法解决**，这是两者思路差异的典型体现。

5.4 完整卡尔曼流程

```
// 预测：协方差对角线加过程噪声（含 P0 上限）
if (P[0] < P0) P[0] += V;
if (P[4] < P0) P[4] += V;
if (P[8] < P0) P[8] += V;

// 新息限幅
vqf_clip(e, 3, -biasClip, biasClip);

// 增益：K = P·R^T·(R·P·R^T + W)^-1
matrix3MultiplyTpsSecond(P, R, K); // K = P·R^T
matrix3Multiply(R, K, K); // K = R·P·R^T
K[0]+=w0; K[4]+=w1; K[8]+=w2; // K += W
matrix3Inv(K, K); // K = (R·P·R^T + W)^-1
matrix3MultiplyTpsFirst(R, K, K); // K = R^T·K
matrix3Multiply(P, K, K); // 最终增益

// 状态更新
bias += K · e;

// 协方差更新：P ← (I - K·R)·P
K = K·R; K = K·P; P -= K;

// 限幅保护
vqf_clip(bias, 3, -biasClip, biasClip);
```

结构上这就是标准卡尔曼滤波的「预测—更新」两步。**但注意：VQF 只对 3 维零偏做卡尔曼**，姿态本身不走协方差传播，而是靠确定性几何构造。这正是它比完整 EKF 轻量的原因。

六、欧拉角与偏航解缠

6.1 四元数转欧拉角（ZYX 序）

```
pitch = asin( 2(q0q2 - q1q3) )
roll = atan2( 2(q0q1 + q2q3), 1 - 2(q1² + q2²) )
yaw = atan2( 2(q1q2 + q0q3), 1 - 2(q2² + q3²) )
```

对应代码 352~354 行。这是标准的 ZYX（先偏航、再俯仰、最后横滚）欧拉角公式。

为什么 pitch 用 asin 而 roll/yaw 用 atan2？ 因为 pitch 的物理范围本来就是 ±90°，asin 的值域正好匹配；而 roll 和 yaw 的范围是 ±180°，需要 atan2 来区分象限。

必须做限幅保护（代码 350~351 行）：浮点误差可能让 sinp 等于 1.0000001，asinf 会返回 NaN，NaN 一旦进入结构体就会污染后续所有输出。

6.2 偏航解缠（Yaw Unwrapping）

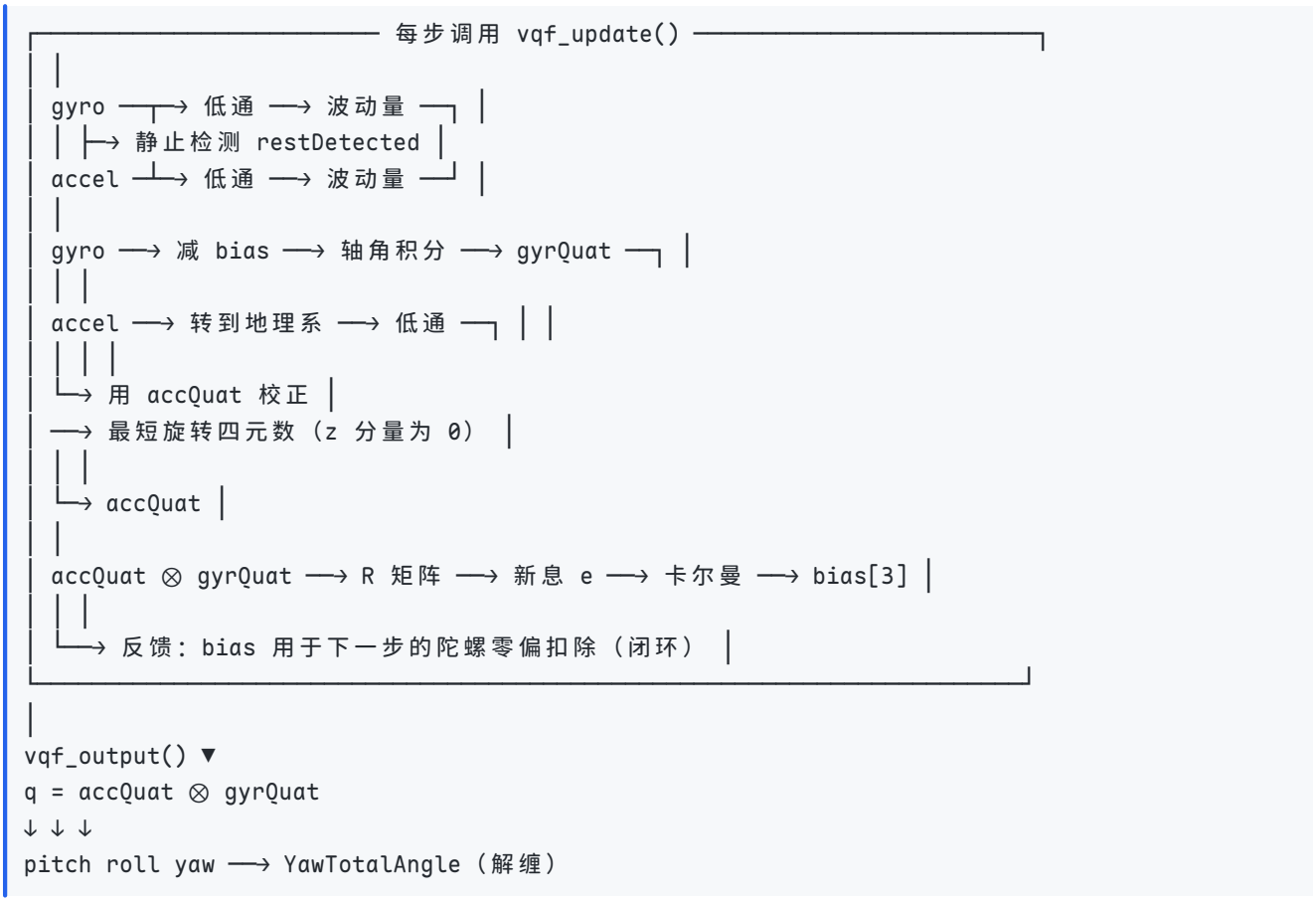
这是 VQF 相对许多实现的一个实用加分项。atan2 输出的 yaw 在 ±180° 处会突变，如果直接送进 PID 做偏航控制，控制器会以为机器人瞬间转了 360°，输出一个巨大的错误指令。

代码 356~366 行的做法是：

```
yaw_diff = yaw - last_yaw
if (yaw_diff > 180) { yaw_diff -= 360; yaw_laps--; }
if (yaw_diff < -180) { yaw_diff += 360; yaw_laps++; }
YawTotalAngle += yaw_diff // 连续累加，可超过 ±360°
last_yaw = yaw
```

这样 YawTotalAngle 就是一个**连续、无跳变、可无限累加**的偏航角，可以直接用于闭环控制。本工程 data_ins.yaw_total 取的就是这个值。

七、整体信号流



注意其中的**反馈环路**：bias 估计出来之后，下一步会从陀螺读数里扣掉它。这就是 VQF 抑制长期漂移的闭环。整个算法因此是一个**自适应的观测器**，而不是开环积分。

VQF_filter.h 逐行解析

头文件定义了滤波器的**全部状态**。理解这个结构体，就理解了 VQF 算法「需要记住什么」——这正是它与 EKF 的差别之一：VQF 的状态是物理量（两个四元数、零偏、滤波器延迟线），而不是抽象的协方差矩阵。

```
1 /**牛逼！
```
文件级文档注释开始。句末的「牛逼！」是作者随手写的感叹，不影响编译。

```
2 * @file vqf_filter.h
```
标注文件名，便于 Doxygen 生成文档。

```
3 * @brief VQF姿态滤波算法头文件
```
一句话说明本文件用途：VQF 姿态滤波算法的头文件。

```
4 */
```
文档注释结束。

```
5
6 #ifndef HORIZON_VQF_FILTER_H
```
头文件保护宏开始。防止本文件被多次 include 时重复定义结构体。

```
7 #define HORIZON_VQF_FILTER_H
```
定义保护宏，与 #ifndef 配对。

```
8
9 #include <stdbool.h>
```
引入 **bool** / true / false，因为下面有开关型成员。

```
10 #include <stdint.h>
```
引入定长整型（uint8_t 等），保证跨平台宽度一致。

```
11
12 typedef float vqf_real_t;
```
单精度别名。所有实时运算量（四元数、角速度、加速度）都用它，因为在 Cortex-M4F 上 float 有硬件 FPU，快且够用。

```
13 typedef double vqf_double_t;
```
双精度别名。滤波器系数、DF2T 状态用它，避免 IIR 递推的累积舍入误差。这是 VQF 的一个细节：系数算得准，长时运行才不会漂。

```
14
15 /**
```
结构体文档注释开始。

```
16 * @brief VQF滤波器结构体
```
说明这是 VQF 滤波器结构体。

```
17 */
```
文档注释结束。

```
18 struct VQF_FILTER_t {
```
结构体定义开始。**一个实例 = 一路完整的姿态估计器**，本例全局只有一个。

```
19 /* ================= 参数（Params）=================
```
参数区分组注释：这些是「用户可以调」的超参数。

```
20 * 这些在 vqf_init() 里设置默认值，可按需调整 */
```
说明这些参数在 vqf_init() 里被赋默认值。

```
21 vqf_real_t tauAcc; /* 加速度计低通滤波时间常数(s)，越大对加速度噪声抑制越强，但动态响应变慢 */
```
加速度计低通时间常数（秒）。默认 3.0 s。这是 VQF 最核心的调参旋钮：越大→重力方向越平滑、越不怕线性加速度，但姿态收敛越慢；越小→跟随快，但晃动和运动加速度会污染姿态。

```
22 bool motionBiasEstEnabled; /* 是否开启"运动中的零偏估计"，开启可在运动时也逐步修正陀螺仪零偏 */
```
是否开启「运动中的零偏估计」。开启后即使在运动状态，也能靠重力方向持续修正陀螺零偏，适合长时间飞行/行驶。

```
23 bool restBiasEstEnabled; /* 是否开启"静止时零偏估计"，静止时用加速度计校准零偏，精度更高 */
```
是否开启「静止时的零偏估计」。检测到静止时用陀螺输出本身作为零偏观测，精度远高于运动估计（因为此时真实角速度就是 0）。

```
24 vqf_real_t biasSigmaInit; /* 零偏估计的初始方差(°/s)，初始不确定度 */
```
零偏初始不确定度 σ（度/秒），默认 0.5。它决定卡尔曼协方差初值 P0，影响零偏估计起步阶段的收敛速度。

```
25 vqf_real_t biasForgettingTime; /* 零偏忘记时间常数(s)，控制零偏估计收敛速度，越小收敛越快 */
```
零偏遗忘时间常数（秒），默认 100。控制过程噪声 V
的大小：越小→系统「以为」零偏变化越快→收敛快但更抖；越大→零偏更稳但适应慢。

```
26 vqf_real_t biasClip; /* 零偏限幅(°/s)，防止零偏估计发散到离谱值 */
```
零偏限幅（度/秒），默认 2.0。防止零偏估计在某些病态输入下发散到离谱值，这是一层硬保护。

```
27 vqf_real_t biasSigmaMotion; /* 运动时零偏过程噪声方差(°/s) */
```
运动估计的过程噪声 σ（度/秒），默认 0.1。反映「运动时零偏可能缓慢变化」这一先验假设。

```
28 vqf_real_t biasVerticalForgettingFactor;/*
竖直方向(重力方向)零偏遗忘因子，越小重力方向零偏收敛越慢 */
```
竖直方向零偏的遗忘因子，默认 0.0001。重力方向（竖直）和水平方向的零偏可观测性完全不同：绕竖直轴的零偏靠重力**永远观测不到**，所以给它一个极小的权重，让它几乎不更新，避免被噪声带偏。这是 VQF 处理可观测性的关键设计。

```
29 vqf_real_t biasSigmaRest; /* 静止时零偏过程噪声方差(°/s) */
```
静止估计的过程噪声 σ（度/秒），默认 0.03。比运动的小，因为静止时观测可信。

```
30 vqf_real_t restMinT; /* 判定"静止"的最短持续时间(s)，静止不足该时长不启用静止零偏估计 */
```
判定「静止」所需的最短持续时间（秒），默认 1.5。防止瞬时低速被误判为静止，否则会把真实转动当成零偏吃进去。

```
31 vqf_real_t restFilterTau; /* 静止检测用的低通滤波时间常数(s) */
```
静止检测用的低通时间常数（秒），默认 0.5。先把陀螺/加速度平滑，再和原始值比较，得到「波动量」。

```
32 vqf_real_t restThGyr; /* 静止检测的陀螺仪角速度阈值(°/s)，低于此值算静止 */
```
静止检测的陀螺角速度阈值（度/秒），默认 2.0。平滑后输出若超过它，就直接判定为「在动」。

```
33 vqf_real_t restThAcc; /* 静止检测的加速度波动阈值(m/s²)，加速度变化小于此值算静止 */
```
静止检测的加速度波动阈值（m/s²），默认
0.5。加速度波动小于它才算静止，这能识别出「匀速直线运动」和「静止」的区别。

```
34
35 /* ================= 系数（Coeffs）=================
```
系数区分组注释：这些由 init 根据上面的参数算出来，用户不要手改。

```
36 * 内部滤波器系数，由 vqf_init() 根据上述参数计算，无需手动改 */
```
再次强调「无需手动改」。

```
37 vqf_real_t gyrTs; /* 陀螺仪采样周期(s)，即传入 vqf_update 的 dt */
```
陀螺采样周期（秒），就是调用 vqf_update 的间隔 dt。本工程是 0.004 s（250 Hz）。

```
38 vqf_real_t accTs; /* 加速度计采样周期(s)，默认与 gyrTs 相同 */
```
加速度采样周期（秒），默认与陀螺相同。VQF 允许两者不同频率。

```
39 vqf_double_t accLpB[3]; /* 加速度低通滤波器分子系数 b0/b1/b2（二阶巴特沃斯） */
```
加速度低通滤波器分子系数 b0/b1/b2。二阶巴特沃斯，由 filterCoeffs() 用双线性变换算出。

```
40 vqf_double_t accLpA[2]; /* 加速度低通滤波器分母系数 a1/a2 */
```
加速度低通分母系数 a1/a2。注意符号约定：这里存的是「正」的 a1、a2，而 filterStep 里是减去它们，即传递函数分母为 $1 + a1 \cdot z^{-1} + a2 \cdot z^{-2}$。

```
41 vqf_real_t biasP0; /* 零偏卡尔曼协方差初值 */
```
零偏卡尔曼协方差初值 P0 = $(\sigma_init \cdot 100)^2$。这个 100 是原版实现引入的数值缩放，它同时缩放 P 和过程噪声
V，因此**在卡尔曼增益里会约掉**，不影响结果，只是让内部数值落在舒适的动态范围内。

```
42 vqf_real_t biasV; /* 零偏卡尔曼过程噪声 */
```
零偏卡尔曼过程噪声 V = $(0.1 \cdot 100)^2 \cdot Ts/T_forget$。每步给协方差对角线加这么多，表示「零偏可能随时间缓慢变化」。

```
43 vqf_real_t biasMotionW; /* 运动零偏估计的测量权重 */
```
运动估计的测量权重 W。由 biasSigmaMotion 和 biasV 共同决定，权重越大表示越信任这次观测。

```
44 vqf_real_t biasVerticalW; /* 重力方向零偏估计的测量权重 */
```
竖直方向零偏的测量权重 = biasMotionW / biasVerticalForgettingFactor。因为遗忘因子只有
1e-4，分母极小，所以这个权重被放大到一万倍——等价于「竖直方向几乎不更新」。

```
45 vqf_real_t biasRestW; /* 静止零偏估计的测量权重 */
```
静止估计的测量权重 W。静止时观测最可信，权重由 biasSigmaRest 决定。

```
46 vqf_double_t restGyrLpB[3]; /* 静止检测陀螺仪低通滤波器分子系数 */
```
静止检测用陀螺低通的分子系数。

```
47 vqf_double_t restGyrLpA[2]; /* 静止检测陀螺仪低通滤波器分母系数 */
```
静止检测用陀螺低通的分母系数。

```
48 vqf_double_t restAccLpB[3]; /* 静止检测加速度低通滤波器分子系数 */
```
静止检测用加速度低通的分子系数。

```
49 vqf_double_t restAccLpA[2]; /* 静止检测加速度低通滤波器分母系数 */
```
静止检测用加速度低通的分母系数。

```
50
51 /* ================= 状态 (State) =================
```
状态区分组注释：内部运行状态，业务代码一般不用读。

```
52 * 滤波器内部运行状态，一般不需要读，给懂算法的人看 */
```
说明这些是算法内部量，给理解算法的人看。

```
53 vqf_real_t gyrQuat[4]; /* 纯陀螺仪积分出的四元数 [w,x,y,z]（只用角速度） */
```
陀螺四元数 gyrQuat。只用角速度积分得到，短时精准、长期漂移。它是最终姿态的「高频骨架」。

```
54 vqf_real_t accQuat[4]; /* 纯加速度计校正出的四元数 [w,x,y,z]（只用重力方向） */
```
加速度四元数
accQuat。它表示「让观测重力对齐真实竖直所需的修正旋转」，只含倾斜信息、不含偏航。它是最终姿态的「低频骨架」。

```
55 bool restDetected; /* 当前是否判定为静止（静置时置 true） */
```
静止标志。由 restMinT / restThGyr / restThAcc 共同判定。

```
56 vqf_real_t lastAccLp[3]; /* 加速度低通滤波后的加速度，用于校正 accQuat */
```
低通后的加速度（在陀螺参考系下）。用于构造 accQuat 的观测。

```
57 vqf_double_t accLpState[6]; /* 加速度低通滤波器的内部状态（3通道 × 2阶） */
```
加速度低通内部状态：3 个通道 × 每通道 2 阶 = 6 个 double。DF2T 结构需要 2 个状态量/通道。

```
58 vqf_real_t lastAccCorrAngularRate; /* 上一次加速度校正产生的角速度(rad/s) */
```
上一步加速度校正产生的等效角速率（rad/s）。等于 acos(重力估计 z
分量)/Ts，也就是「这一步把姿态拽正了多少」。它同时是运动零偏估计的观测量来源。

```
59 vqf_real_t bias[3]; /* 当前估计的陀螺仪零偏(rad/s)，vqf_update 内部会减去它 */
```
当前估计的陀螺零偏（rad/s）。每一步积分前会把它从原始陀螺里减掉，这是 VQF 抑制长期漂移的核心。

```
60 vqf_real_t biasP[9]; /* 零偏卡尔曼协方差矩阵（3×3） */
```
零偏卡尔曼协方差矩阵 3×3，按行主序存 9 个数。对角线是三个轴的方差，非对角线是轴间相关性。

```
61 vqf_double_t motionBiasEstRLpState[18]; /* 运动零偏估计中 R 矩阵滤波状态（9通道 × 2阶） */
```
运动零偏估计中 R 矩阵的低通状态：9 通道 × 2 阶 = 18 个 double。

```
62 vqf_double_t motionBiasEstBiasLpState[4];/* 运动零偏估计中 bias 滤波状态（2通道 × 2阶） */
```
运动零偏估计中 biasLp 的低通状态：2 通道 × 2 阶 = 4 个 double。注意只有 2
通道——因为第三个（竖直）方向不可观测，被刻意排除在外。

```
63 vqf_real_t restLastSquaredDeviations[2];/* 静止检测的累计方差：[0]陀螺仪 [1]加速度 */
```
静止检测的累计方差：[0] 陀螺、[1] 加速度。即「原始值 − 平滑值」的平方和，越大说明波动越大。

```
64 vqf_real_t restT; /* 当前已持续"疑似静止"的累计时间(s) */
```
已持续「疑似静止」的时间（秒）。超过 restMinT 才真正置 restDetected。

```
65 vqf_real_t restLastGyrLp[3]; /* 静止检测陀螺仪滤波输出 */
```
静止检测陀螺低通的输出。

```
66 vqf_double_t restGyrLpState[6]; /* 静止检测陀螺仪滤波器内部状态 */
```
静止检测陀螺低通的内部状态。

```
67 vqf_real_t restLastAccLp[3]; /* 静止检测加速度滤波输出 */
```
静止检测加速度低通的输出。

```
68 vqf_double_t restAccLpState[6]; /* 静止检测加速度滤波器内部状态 */
```
静止检测加速度低通的内部状态。

```
69
```

```
70 /* ================= 输出（Outputs）=================
```
输出区分组注释：业务代码主要用这里的字段。

```
71 * 业务代码主要用这里的字段，由 vqf_output() 每次更新 */
```
说明这些字段由 vqf_output() 每步刷新。

```
72 float q[4]; /* 【最终四元数】[w,x,y,z]，机体姿态的完整表示 */
```
最终四元数 [w,x,y,z] = accQuat ⊗ gyrQuat。姿态的完整表示，没有万向锁问题，适合做插值和控制。

```
73 float pitch, roll, yaw; /* 【欧拉角】俯仰角/横滚角/偏航角（度，yaw 范围 ±180） */
```
欧拉角（度）：pitch 俯仰、roll 横滚、yaw 偏航。yaw 范围 ±180°，因为是 atan2 算出来的。欧拉角只适合「看」和「显示」，直接拿去做控制会踩万向锁。

```
74 float yaw_laps; /* 偏航角跨过 ±180 的次数（绕圈计数，每转一圈 ±1） */
```
偏航跨越 ±180° 的圈数。每转一圈 ±1。

```
75 float last_yaw; /* 上一次的 yaw，内部用于计算累计偏航 */
```
上一次的 yaw。内部用于算差值，判断有没有跨过 ±180°。

```
76 float YawTotalAngle; /* 【累计偏航角】(度)，可跨过 ±180 连续累加，适合做控制 */
```
累计偏航角（度）——VQF 一个很实用的额外输出。它把 ±180° 的跳变解开成连续累加的角度，可以超过 360°、−360°，**适合直接做偏航闭环控制**。本工程 data_ins.yaw_total 就取这个值。

```
77 float rMat[3][3]; /* 【旋转矩阵】机体坐标系转地理坐标系（当前未更新，备用） */
```
旋转矩阵 3×3，机体→地理。本实现中**没有真正更新它**，只是占位备用。如果要给控制用矩阵形式，需要自己从 q 换算。

```
78 };
79
80 extern struct VQF_FILTER_t vqf_filter;
```
声明全局唯一实例，定义在 .c 第 21 行。**extern** 表示「别处有定义」。

```
81
82 void vqf_init(struct VQF_FILTER_t *f, float dt);
```
声明初始化函数：传入采样周期 dt（秒），完成参数赋值与状态复位。

```
83 void vqf_update(struct VQF_FILTER_t *f, float gx, float gy, float gz, float ax, float ay, float
az);
```
声明核心更新函数：陀螺三轴 + 加速度三轴（单位分别是 rad/s 和 m/s²）。注意**没有磁力计参数**——本实现是 6 轴 VQF，偏航靠陀螺积分维持。

```
84 void vqf_output(struct VQF_FILTER_t *f);
```
声明输出函数：由两个四元数合成最终四元数，并换算欧拉角、解缠偏航角。

```
85
86 #endif //HORIZON_VQF_FILTER_H
```
头文件保护宏结束。

```
87
```

VQF_filter.c 逐行解析

实现文件共 367 行，分四层：**基础数学工具**（四元数、矩阵、滤波器）、**滤波器系数设计**、**初始化**、**核心更新**。下面逐行说明每一行的作用。

```
1 /**
```
文件级文档注释开始。

```
2 * @file vqf_filter.c
```
标注当前文件名。

```
3 * @brief VQF姿态滤波算法
```
一句话描述：VQF 姿态滤波算法。

```
4 * @note
```
备用注释字段（内容为空）。

```
5 */
```
文档注释结束。

```
6 #include "VQF_filter.h"
```
引入本模块头文件，拿到结构体定义与三个函数声明。

```
7 #include <float.h>
```
引入 float.h：为了用 **FLT_EPSILON**（float 的最小可分辨差值）。

```
8 #include <math.h>
```
引入 math.h：提供 sqrtf / cosf / sinf / tanf / acosf / asinf / atan2f / isnan / NAN。

```
9 #include <string.h>
```
引入 string.h：提供 memset / memcpy。

```
10
11 #define EPS FLT_EPSILON
```
全局数值容差。凡是判断「接近 0」都用它。FLT_EPSILON ≈ 1.19e-7，即 float 能达到的最小相对精度。

```
12 #define M_PIf 3.14159265358979323846f
```
单精度 π。math.h 的 M_PI 是 double，这里加 f 后缀，避免常数运算把整个表达式提升成 double。

```
13
14 #ifndef M_SQRT2
```
条件编译：如果系统没定义 √2 才定义。

```
15 #define M_SQRT2 1.41421356237309504880
```
√2 ≈ 1.41421356。二阶巴特沃斯滤波器的标准系数必须用到它（Q = 1/√2）。

```
16 #endif
```
条件编译结束。

```
17 #ifndef RAD2DEG
```
条件编译：如果没定义「弧度→角度」的换算系数。

```
18 #define RAD2DEG 57.29577951308232f
```
180/π ≈ 57.29578。把弧度乘上它就得到角度。

```
19 #endif
```
条件编译结束。

```
20
21 struct VQF_FILTER_t vqf_filter;
```
定义全局唯一的滤波器实例。.h 里的 extern 指向这里。静态存储期变量默认清零，所以初值全为 0。

```
22
23
24 static float vqf_square(float x) { return x * x; }
```
平方辅助函数。static 表示只在本文件可见（不会和其他文件的同名字符冲突）。

```
25 static float vqf_max(float a, float b) { return a > b ? a : b; }
```
取较大值辅助函数。用于把 biasVerticalForgettingFactor 托底，防止除零。

```
26
27 static vqf_real_t vqf_norm(const vqf_real_t vec[], size_t N) {
```
求向量欧氏范数（模长）的辅助函数。vec 是输入向量，N 是元素个数。

```
28 vqf_real_t s = 0;
```
累加器清零。

```
29 for(size_t i = 0; i < N; i++) s += vec[i]*vec[i];
```
循环累加各分量的平方：s = Σ vec[i]²。

```
30 return sqrtf(s);
```
开平方得到模长 **||vec||**。用 sqrtf 而非 sqrt，走单精度路径。

```
31 }
```
函数结束。

```
32
33 static void vqf_normalize(vqf_real_t vec[], size_t N) {
```
向量归一化（单位化）：把向量就地改成方向不变、模长为 1 的向量。

```
34 vqf_real_t n = vqf_norm(vec, N);
```
先算出当前模长 n。

```
35 if (n < EPS) return;
```
关键保护：如果模长接近 0，直接返回，什么都不做。否则除以 ~0 会得到 inf/NaN，进而污染整个滤波器状态。

```
36 for(size_t i = 0; i < N; i++) vec[i] /= n;
```
逐元素除以模长，完成单位化。

```
37 }
```
函数结束。

```
38
39 static void vqf_clip(vqf_real_t vec[], size_t N, vqf_real_t min, vqf_real_t max) {
```
向量限幅：把每个元素夹在 [min, max] 区间内。用于零偏和误差的硬约束。

```
40 for(size_t i = 0; i < N; i++) {
```
遍历 N 个元素。

```
41 if (vec[i] < min) vec[i] = min;
```
若小于下限，钳到下限。

```
42 else if (vec[i] > max) vec[i] = max;
```
否则若大于上限，钳到上限。

```
43 }
```
循环结束。

```
44 }
```
函数结束。

```
45
46 static void vqf_quatMultiply(const vqf_real_t q1[4], const vqf_real_t q2[4], vqf_real_t out[4])
{
```
四元数乘法 out = q1 ⊗ q2。采用 [w, x, y, z] 存储顺序，q1[0]=w、q1[1]=x…。四元数乘法对应「旋转的复合」：先做 q2 的旋转、再做 q1 的旋转。

```
47 vqf_real_t w = q1[0]*q2[0] - q1[1]*q2[1] - q1[2]*q2[2] - q1[3]*q2[3];
```
实部 w：w1w2 − x1x2 − y1y2 − z1z2。

```
48 vqf_real_t x = q1[0]*q2[1] + q1[1]*q2[0] + q1[2]*q2[3] - q1[3]*q2[2];
```
x 分量：w1x2 + x1w2 + y1z2 − z1y2。

```
49 vqf_real_t y = q1[0]*q2[2] - q1[1]*q2[3] + q1[2]*q2[0] + q1[3]*q2[1];
```
y 分量：w1y2 − x1z2 + y1w2 + z1x2。

```
50 vqf_real_t z = q1[0]*q2[3] + q1[1]*q2[2] - q1[2]*q2[1] + q1[3]*q2[0];
```
z 分量：w1z2 + x1y2 − y1x2 + z1w2。四行合起来就是标准哈密顿积。

```
51 out[0] = w; out[1] = x; out[2] = y; out[3] = z;
```
把结果写回输出数组。注意 out 理论上不能与 q1/q2 是同一块内存，但本文件里 vqf_quatMultiply(f->gyrQuat, gyrStepQuat, f->gyrQuat) 正是原地更新！它之所以安全，是因为四个分量已全部算进局部变量 w/x/y/z，最后才统一写回。

```
52 }
```
函数结束。

```
53
54 static void vqf_quatRotate(const vqf_real_t q[4], const vqf_real_t v[3], vqf_real_t out[3]) {
```
用四元数旋转向量 out = R(q)·v。先把 q 展开成旋转矩阵，再做矩阵乘向量，比对 v 做两次四元数乘法快。

```
55 vqf_real_t x = (1 - 2*q[2]*q[2] - 2*q[3]*q[3])*v[0] + 2*v[1]*(q[2]*q[1] - q[0]*q[3]) +
2*v[2]*(q[0]*q[2] + q[3]*q[1]);
```
结果 x 分量 = 旋转矩阵第一行 · v。括号里的三个系数就是矩阵的 R00、R01、R02。

```
56 vqf_real_t y = 2*v[0]*(q[0]*q[3] + q[2]*q[1]) + v[1]*(1 - 2*q[1]*q[1] - 2*q[3]*q[3]) +
2*v[2]*(q[2]*q[3] - q[1]*q[0]);
```
结果 y 分量 = 旋转矩阵第二行 · v。

```
57 vqf_real_t z = 2*v[0]*(q[3]*q[1] - q[0]*q[2]) + 2*v[1]*(q[0]*q[1] + q[3]*q[2]) + v[2]*(1 -
2*q[1]*q[1] - 2*q[2]*q[2]);
```
结果 z 分量 = 旋转矩阵第三行 · v。

```
58 out[0] = x; out[1] = y; out[2] = z;
```
写回输出。

```
59 }
```
函数结束。

```
60
61 static void filterCoeffs(vqf_real_t tau, vqf_real_t Ts, vqf_double_t outB[], vqf_double_t
outA[]) {
```
二阶巴特沃斯低通滤波器系数设计。输入时间常数 tau 和采样周期 Ts，输出分子 b[3] 和分母 a[2]。这是本文件里所有低通滤波器的地基。

```
62 vqf_double_t fc = (M_SQRT2 / (2.0*M_PIf))/(vqf_double_t)(tau);
```
由时间常数换算截止频率。注意用的是 **√2/(2π)** 而不是常见的 1/(2π)：这是为了让二阶滤波器的时间响应与「同 tau 的一阶滤波器」等效，是 VQF 原作者的约定，比 1/(2π) 高出约 41%。

```
63 vqf_double_t C = tanf(M_PIf*fc*(vqf_double_t)(Ts));
```
预畸变：C = tan(π·fc·Ts)。双线性变换会把模拟频率「压缩」，这一步提前补偿，保证数字滤波器在 fc 处的增益准确。

```
64 vqf_double_t D = C*C + M_SQRT2*C + 1;
```
中间量 D = C² + √2·C + 1，即分母多项式的常数部分。

```
65 vqf_double_t b0 = C*C/D;
```
分子系数基准 b0 = C²/D。

```
66 outB[0] = b0; outB[1] = 2*b0; outB[2] = b0;
```
分子系数：b = [b0, 2b0, b0]。对称形式正是巴特沃斯的标志。

```
67 outA[0] = 2*(C*C-1)/D;
```
分母系数 a1 = 2(C²−1)/D。**注意存的是正号**，filterStep 里用减法，等价于标准形式 1 + a1·z^{-1} + a2·z^{-2}。

```
68 outA[1] = (1-M_SQRT2*C+C*C)/D;
```
分母系数 a2 = (1 − √2·C + C²)/D。C→0 时 a1→−2、a2→1，极点趋近 z=1，即带宽趋近 0，符合直觉。

```
69 }
```
函数结束。此时四个系数保证直流增益恰为 1（b0+b1+b2 = 1+a1+a2）。

```
70
71 static void filterInitialState(vqf_real_t x0, const vqf_double_t b[3], const vqf_double_t a[2],
vqf_double_t out[]) {
```
计算滤波器的初始状态，使滤波器在「输入恒为 x0」时输出直接就等于 x0，避免启动瞬间产生一个大暂态。这是 VQF 平滑启动的关键技巧。

```
72 out[0] = x0*(1 - b[0]);
```
第一级状态：s0 = x0·(1 − b0)。由 DF2T 的 y = b0·x + s0 反推，要让 y = x0 就必须 s0 = x0 − b0·x0。

```
73 out[1] = x0*(b[2] - a[1]);
```
第二级状态：s1 = x0·(b2 − a2)。这是 DF2T 稳态方程 s1 = b2·x − a2·y 的直接结果。

```
74 }
```
函数结束。

```
75
76 static vqf_real_t filterStep(vqf_real_t x, const vqf_double_t b[3], const vqf_double_t a[2],
vqf_double_t state[2]) {
```
二阶 IIR 单步递推，采用「直接II型转置」（DF2T）结构。x 是输入，b/a 是系数，state[2] 是本通道的两个状态，返回滤波输出。

```
77 vqf_double_t y = b[0]*x + state[0];
```
输出 y = b0·x + s0。这里的 s0 是**上一步**留下的状态。

```
78 state[0] = b[1]*x - a[0]*y + state[1];
```
更新第一级状态：s0 ← b1·x − a1·y + s1。

```
79 state[1] = b[2]*x - a[1]*y;
```
更新第二级状态：s1 ← b2·x − a2·y。

```
80 return y;
```
返回本步输出。注意返回类型是 float，而内部状态是 double——「窄进宽出」，计算精度不丢。

```
81 }
```
函数结束。

```
82
83 static void filterVec(const vqf_real_t x[], size_t N, vqf_real_t tau, vqf_real_t Ts, const
vqf_double_t b[3],
```
对向量做低通滤波。x 是 N 维输入，out 是 N 维输出，state 是 2N 个状态（每通道 2 个）。

```
84 const vqf_double_t a[2], vqf_double_t state[], vqf_real_t out[]) {
85 if (isnan(state[0])) {
```
启动相位的判定：靠「state[0] 是不是 NaN」来判断。vqf_init 会把所有状态置为 NAN，所以第一次调用必定走进这个分支。用 NaN 当哨兵值比额外加一个 bool 标志更省内存。

```
86 if (isnan(state[1])) {
```
如果 state[1] 也是 NaN，说明这是**绝对第一次**调用。

```
87 state[1] = 0;
```
把计数器清零。

```
88 for(size_t i = 0; i < N; i++) state[2+i] = 0;
```
把 N 个累加器清零。这里复用了 state 数组的 [2..N+1] 位置做累加器，它们此刻还没被当状态用。

```
89 }
```
if 结束。

```
90 state[1]++;
```
计数器自增。state[1] 在启动阶段被临时征用为「已经积累了多少个样本」。

```
91 for (size_t i = 0; i < N; i++) {
```
遍历 N 个通道。

```
92 state[2+i] += x[i];
```
把这步的输入累加进累加器。

```
93 out[i] = state[2+i]/state[1];
```
输出 = 累加和 / 样本数，也就是到目前为止的**算术平均**。启动阶段用滑动平均代替 IIR，避免初值造成的冲击。

```
94 }
```
循环结束。

```
95 if (state[1]*Ts >= tau) {
```
启动阶段结束条件：累计时长 state[1]·Ts 达到 tau，也就是「已经观察够一个时间常数了」。

```
96 for(size_t i = 0; i < N; i++) filterInitialState(out[i], b, a, state+2*i);
```
用当前均值 out[i] 反算出 IIR 的稳态初值，正式把滤波器状态初始化好。

```
97 }
```
if 结束。

```
98 return;
```
启动阶段直接返回，不执行 IIR。

```
99 }
```
if 结束。

```
100 for (size_t i = 0; i < N; i++) {
```
正常阶段：对每个通道执行一次 IIR 递推。

```
101 out[i] = filterStep(x[i], b, a, state+2*i);
```
对第 i 通道调 filterStep，状态偏移 2*i。

```
102 }
```
循环结束。

```
103 }
```
函数结束。

```
104
105 static void matrix3SetToScaledIdentity(vqf_real_t scale, vqf_real_t out[9]) {
```
把 3×3 矩阵设成「标量乘单位阵」。scale 是对角线值，out 是 9 元素输出。

```
106 memset(out, 0, 9*sizeof(vqf_real_t));
```
先整体清零。

```
107 out[0] = out[4] = out[8] = scale;
```
把三个对角线元素设为 scale。

```
108 }
```
函数结束。

```
109
110 static void matrix3Multiply(const vqf_real_t in1[9], const vqf_real_t in2[9], vqf_real_t
out[9]) {
```
3×3 矩阵乘法 out = in1 · in2。

```
111 vqf_real_t tmp[9];
```
用临时数组存结果，**避免 in1 或 in2 与 out 是同一块内存时被提前覆盖**。本文件后面大量用到原地乘法，靠的就是这个保护。

```
112 tmp[0] = in1[0]*in2[0] + in1[1]*in2[3] + in1[2]*in2[6]; tmp[1] = in1[0]*in2[1] + in1[1]*in2[4]
+ in1[2]*in2[7]; tmp[2] = in1[0]*in2[2] + in1[1]*in2[5] + in1[2]*in2[8];
```
结果第 1 行：in1 第 1 行 · in2 的三列。

```
113 tmp[3] = in1[3]*in2[0] + in1[4]*in2[3] + in1[5]*in2[6]; tmp[4] = in1[3]*in2[1] + in1[4]*in2[4]
+ in1[5]*in2[7]; tmp[5] = in1[3]*in2[2] + in1[4]*in2[5] + in1[5]*in2[8];
```
结果第 2 行。

```
114 tmp[6] = in1[6]*in2[0] + in1[7]*in2[3] + in1[8]*in2[6]; tmp[7] = in1[6]*in2[1] + in1[7]*in2[4]
+ in1[8]*in2[7]; tmp[8] = in1[6]*in2[2] + in1[7]*in2[5] + in1[8]*in2[8];
```
结果第 3 行。

```
115 memcpy(out, tmp, sizeof(tmp));
```
把临时结果拷回 out。

```
116 }
```
函数结束。

```
117
118 static void matrix3MultiplyTpsFirst(const vqf_real_t in1[9], const vqf_real_t in2[9],
vqf_real_t out[9]) {
```
矩阵乘法（转置左操作数） out = $in1^{T}$ · in2。卡尔曼公式里 $A \cdot B^{T}$ 和 $A^{T} \cdot B$ 频繁出现，单独写出来能省一次显式转置。

```
119 vqf_real_t tmp[9];
```
临时数组。

```
120 tmp[0] = in1[0]*in2[0] + in1[3]*in2[3] + in1[6]*in2[6]; tmp[1] = in1[0]*in2[1] + in1[3]*in2[4]
+ in1[6]*in2[7]; tmp[2] = in1[0]*in2[2] + in1[3]*in2[5] + in1[6]*in2[8];
```
结果第 1 行 = in1 的**第一列**乘 in2 各行，这就是转置的效果。

```
121 tmp[3] = in1[1]*in2[0] + in1[4]*in2[3] + in1[7]*in2[6]; tmp[4] = in1[1]*in2[1] + in1[4]*in2[4]
+ in1[7]*in2[7]; tmp[5] = in1[1]*in2[2] + in1[4]*in2[5] + in1[7]*in2[8];
```

结果第 2 行 = in1 第二列。

```
122 tmp[6] = in1[2]*in2[0] + in1[5]*in2[3] + in1[8]*in2[6]; tmp[7] = in1[2]*in2[1] + in1[5]*in2[4]
+ in1[8]*in2[7]; tmp[8] = in1[2]*in2[2] + in1[5]*in2[5] + in1[8]*in2[8];
```

结果第 3 行 = in1 第三列。

```
123 memcpy(out, tmp, sizeof(tmp));
```

写回。

```
124 }
```

函数结束。

```
125
126 static void matrix3MultiplyTpsSecond(const vqf_real_t in1[9], const vqf_real_t in2[9],
vqf_real_t out[9]) {
```

矩阵乘法（转置右操作数） out = in1 · $in2^T$。

```
127 vqf_real_t tmp[9];
```

临时数组。

```
128 tmp[0] = in1[0]*in2[0] + in1[1]*in2[1] + in1[2]*in2[2]; tmp[1] = in1[0]*in2[3] + in1[1]*in2[4]
+ in1[2]*in2[5]; tmp[2] = in1[0]*in2[6] + in1[1]*in2[7] + in1[2]*in2[8];
```

结果第 1 行 = in1 第一行 与 in2 的**各行**做点积（即与 $in2^T$ 的列）。

```
129 tmp[3] = in1[3]*in2[0] + in1[4]*in2[1] + in1[5]*in2[2]; tmp[4] = in1[3]*in2[3] + in1[4]*in2[4]
+ in1[5]*in2[5]; tmp[5] = in1[3]*in2[6] + in1[4]*in2[7] + in1[5]*in2[8];
```

结果第 2 行。

```
130 tmp[6] = in1[6]*in2[0] + in1[7]*in2[1] + in1[8]*in2[2]; tmp[7] = in1[6]*in2[3] + in1[7]*in2[4]
+ in1[8]*in2[5]; tmp[8] = in1[6]*in2[6] + in1[7]*in2[7] + in1[8]*in2[8];
```

结果第 3 行。

```
131 memcpy(out, tmp, sizeof(tmp));
```

写回。

```
132 }
```

函数结束。

```
133
134 static bool matrix3Inv(const vqf_real_t in[9], vqf_real_t out[9]) {
```

3×3 矩阵求逆，用「伴随矩阵 / 行列式」的解析公式。相比高斯消元，它没有循环、没有分支，在 MCU 上快得多。

```
135 vqf_double_t A = in[4]*in[8] - in[5]*in[7]; vqf_double_t D = in[2]*in[7] - in[1]*in[8];
vqf_double_t G = in[1]*in[5] - in[2]*in[4];
```

计算第一行代数余子式 A、D、G。

```
136 vqf_double_t B = in[5]*in[6] - in[3]*in[8]; vqf_double_t E = in[0]*in[8] - in[2]*in[6];
vqf_double_t H = in[2]*in[3] - in[0]*in[5];
```

计算第二行代数余子式 B、E、H。

```
137 vqf_double_t C = in[3]*in[7] - in[4]*in[6]; vqf_double_t F = in[1]*in[6] - in[0]*in[7];
vqf_double_t I = in[0]*in[4] - in[1]*in[3];
```

计算第三行代数余子式 C、F、I。

```
138 vqf_double_t det = in[0]*A + in[1]*B + in[2]*C;
```

行列式 det = a00·A + a01·B + a02·C（按第一行展开）。用 double 累加，防止 3 项相消造成精度损失。

```
139 if (det >= -EPS && det <= EPS) {
```

奇异判定：行列式绝对值小于 EPS 就认为矩阵不可逆。

```
140 memset(out, 0, 9*sizeof(vqf_real_t));
```

输出矩阵清零。

```
141 return false;
```

返回 false 表示失败。

```
142 }
```

if 结束。

```
143 out[0] = A/det; out[1] = D/det; out[2] = G/det; out[3] = B/det; out[4] = E/det; out[5] = H/det;
out[6] = C/det; out[7] = F/det; out[8] = I/det;
```

伴随矩阵除以行列式。**注意元素的转置排布**：out[1] 拿的是 D（不是 B），这正是「伴随矩阵」的定义——代数余子式矩阵的转置。

```
144 return true;
```

返回 true 表示成功。

```
145 }
```

函数结束。

```
146
147
148 void vqf_init(struct VQF_FILTER_t *f, float dt)
```

初始化滤波器。f 是实例指针，dt 是采样周期（秒）。本工程在 INS.c 里传 0.004f，即 250 Hz。

```
149 {
150 // 初始化参数
```

参数赋值区注释。

```
151 f->tauAcc = 3.0f;
```

加速度低通时间常数 = 3.0 s。这是 VQF 最影响手感的参数，3 秒意味着重力方向要花好几秒才能完全「跟上」姿态变化。

```
152 f->motionBiasEstEnabled = true;
```

开启运动零偏估计。

```
153 f->restBiasEstEnabled = true;
```

开启静止零偏估计。

```
154 f->biasSigmaInit = 0.5f;
```

零偏初始 σ = 0.5 °/s。

```
155 f->biasForgettingTime = 100.0f;
```

零偏遗忘时间 = 100 s。

```
156 f->biasClip = 2.0f;
```

零偏限幅 = 2.0 °/s。超过这个值的零偏估计会被硬钳住。

```
157 f->biasSigmaMotion = 0.1f;
```

运动过程噪声 σ = 0.1 °/s。

```
158 f->biasVerticalForgettingFactor = 0.0001f;
```

竖直方向遗忘因子 = 0.0001。这个极小的值让「绕重力轴」的零偏几乎不被更新，因为该方向本来就不被重力观测到。

```
159 f->biasSigmaRest = 0.03f;
```

静止过程噪声 σ = 0.03 °/s。

```
160 f->restMinT = 1.5f;
```

静止判定最短时长 = 1.5 s。必须有 1.5 秒持续不动才启用静止零偏估计。

```
161 f->restFilterTau = 0.5f;
```

静止检测低通 tau = 0.5 s。

```
162 f->restThGyr = 2.0f;
```

静止检测陀螺阈值 = 2.0 °/s。

```
163 f->restThAcc = 0.5f;
```

静止检测加速度波动阈值 = 0.5 m/s²。

```
164
165 // 时间常数
```

时间常数区注释。

```
166 f->gyrTs = dt;
```

记录陀螺采样周期。

```
167 f->accTs = dt;
```

加速度采样周期与之相同。

```
168
```

```
169 // 计算滤波器系数
```
系数计算区注释。

```
170 filterCoeffs(f->tauAcc, f->accTs, f->accLpB, f->accLpA);
```
算加速度低通的巴特沃斯系数，用 tauAcc 和 accTs。

```
171 f->biasP0 = vqf_square(f->biasSigmaInit*100.0f);
```
零偏协方差初值 P0 = (0.5·100)² = 2500。

```
172 f->biasV = vqf_square(0.1f*100.0f) * f->accTs / f->biasForgettingTime;
```
过程噪声 V = (0.1·100)²·Ts/100。含 Ts 说明它是「每步」的增量，除以遗忘时间表示零偏变化的时间尺度。

```
173
174 vqf_real_t pMotion = vqf_square(f->biasSigmaMotion*100.0f);
```
运动观测方差 pMotion = (0.1·100)² = 100。

```
175 f->biasMotionW = vqf_square(pMotion) / f->biasV + pMotion;
```
运动测量权重 W = pMotion²/V + pMotion。这个式子来自把「过程噪声」和「观测噪声」合成为等效权重，是原版 VQF 的推导结果。

```
176 f->biasVerticalW = f->biasMotionW / vqf_max(f->biasVerticalForgettingFactor, 1e-10f);
```
竖直权重 = biasMotionW / max(factor, 1e-10)。factor 只有
1e-4，所以竖直权重被放大约一万倍，效果是让卡尔曼增益在那个方向上趋近 0——**等于「不更新」**。max() 是防止用户把 factor 设成 0 导致除零。

```
177
178 vqf_real_t pRest = vqf_square(f->biasSigmaRest*100.0f);
```
静止观测方差 pRest = (0.03·100)² = 9。

```
179 f->biasRestW = vqf_square(pRest) / f->biasV + pRest;
```
静止测量权重 W = pRest²/V + pRest。

```
180
181 filterCoeffs(f->restFilterTau, f->gyrTs, f->restGyrLpB, f->restGyrLpA);
```
算静止检测陀螺低通系数。

```
182 filterCoeffs(f->restFilterTau, f->accTs, f->restAccLpB, f->restAccLpA);
```
算静止检测加速度低通系数。

```
183
184 // 重置状态
```
状态复位区注释。

```
185 f->gyrQuat[0] = 1; f->gyrQuat[1] = 0; f->gyrQuat[2] = 0; f->gyrQuat[3] = 0;
```
陀螺四元数置为单位四元数 [1,0,0,0]。表示「初始姿态为水平、朝向原点」，这是算法约定的初始朝向。

```
186 f->accQuat[0] = 1; f->accQuat[1] = 0; f->accQuat[2] = 0; f->accQuat[3] = 0;
```
加速度四元数同样置为单位四元数。

```
187 f->restDetected = false;
```
静止标志初始为 false。

```
188 memset(f->lastAccLp, 0, sizeof(f->lastAccLp));
```
低通后的加速度清零。

```
189 memset(f->bias, 0, sizeof(f->bias));
```
零偏估计清零——**假设开机时没有零偏**，之后由卡尔曼逐步学出来。

```
190 matrix3SetToScaledIdentity(f->biasP0, f->biasP);
```
协方差矩阵置为 P0•I，即三个轴独立、方差都是 P0。

```
191
192 for (size_t i = 0; i < 6; i++) {
```
遍历 6 个 IIR 状态。

```
193 f->accLpState[i] = NAN;
```
加速度低通状态置 NaN——这就是前面 filterVec 用来识别「首次调用」的哨兵。

```
194 f->restGyrLpState[i] = NAN;
```
静止检测陀螺低通状态置 NaN。

```
195 f->restAccLpState[i] = NAN;
```
静止检测加速度低通状态置 NaN。

```
196 }
```
循环结束。

```
197 for (size_t i = 0; i < 18; i++) f->motionBiasEstRLpState[i] = NAN;
```
运动零偏估计的 R 低通状态（18 个）全部置 NaN。

```
198 for (size_t i = 0; i < 4; i++) f->motionBiasEstBiasLpState[i] = NAN;
```
运动零偏估计的 biasLp 低通状态（4 个）全部置 NaN。

```
199 for (size_t i = 0; i < 3; i++) {
```
遍历 3 个轴。

```
200 f->restLastGyrLp[i] = NAN;
```
静止检测的陀螺输出置 NaN，这也是给后续逻辑当「尚未初始化」的标记。

```
201 f->restLastAccLp[i] = 0.0f;
```
静止检测加速度输出置 0。

```
202 }
```
循环结束。

```
203 f->restLastSquaredDeviations[0] = f->restLastSquaredDeviations[1] = 0.0f;
```
两个累计方差清零。

```
204 f->restT = 0.0f;
```
疑似静止计时清零。

```
205
206 f->pitch = 0; f->roll = 0; f->yaw = 0;
```
欧拉角输出清零。

```
207 f->yaw_laps = 0; f->last_yaw = 0; f->YawTotalAngle = 0;
```
偏航解缠相关变量清零：圈数、上次 yaw、累计角度。如果不做这一步，重新 init 后累计角度会从旧值接着算。

```
208 }
```
初始化函数结束。

```
209
210 void vqf_update(struct VQF_FILTER_t *f, float gx, float gy, float gz, float ax, float ay, float
az)
```
核心更新函数。输入六轴：陀螺 gx/gy/gz（rad/s）与加速度 ax/ay/az（m/s²）。每来一批 IMU 数据调一次。

```
211 {
212 vqf_real_t gyr[3] = {gx, gy, gz};
```
把三个陀螺通道拷进局部数组，方便后面按向量处理。

```
213 vqf_real_t acc[3] = {ax, ay, az};
```
加速度同理。

```
214
215 /* ----------------- 更新陀螺仪 ----------------- */
```
陀螺处理区注释。

```
216 if (f->restBiasEstEnabled) {
```
只有在开启静止零偏估计时才做这套检测。

```
217 filterVec(gyr, 3, f->restFilterTau, f->gyrTs, f->restGyrLpB, f->restGyrLpA,
```
对陀螺做低通，结果存进 restLastGyrLp。注意 gyr 原值仍保留在局部数组里没被改，因为后面还要用。

```
218 f->restGyrLpState, f->restLastGyrLp);
```
（上一行的续行）状态与输出指针。

```
219 f->restLastSquaredDeviations[0] = vqf_square(gyr[0] - f->restLastGyrLp[0])
```
算「波动量」：原始陀螺与低通值之差的平方和。静止时陀螺输出稳定，低通值几乎等于原值，这个平方和就很小；一旦真的在转，低通跟不上，差值立刻变大。

```
220 + vqf_square(gyr[1] - f->restLastGyrLp[1]) + vqf_square(gyr[2] - f->restLastGyrLp[2]);
```
（续行）三个轴平方相加。

```
221
222 vqf_real_t biasClip = f->biasClip*(M_PIf/180.0f);
```
把限幅值从「度/秒」换算成弧度/秒，供后面比较。

```
223 if (f->restLastSquaredDeviations[0] >= vqf_square(f->restThGyr*(M_PIf/180.0f))
```
判定「不在静止」：波动量超过阈值 restThGyr。

```
224 || fabsf(f->restLastGyrLp[0]) > biasClip || fabsf(f->restLastGyrLp[1]) > biasClip
```
或者低通后的任意轴角速度绝对值超过零偏限幅——意思是「平滑后仍在明显转动」。

```
225 || fabsf(f->restLastGyrLp[2]) > biasClip) {
```
（续行）第三个轴。

```
226 f->restT = 0.0;
```
计时清零。任何一次「在动」都会把静止累计时间打断。

```
227 f->restDetected = false;
```
静止标志清 false。

```
228 }
```
if 结束。

```
229 }
```
条件块结束。

```
230
231 vqf_real_t gyrNoBias[3] = {gyr[0]-f->bias[0], gyr[1]-f->bias[1], gyr[2]-f->bias[2]};
```
扣除零偏：gyrNoBias = 原始陀螺 − 当前零偏估计。这是把长期漂移消掉的关键一步。

```
232 vqf_real_t gyrNorm = vqf_norm(gyrNoBias, 3);
```
求角速度向量的模长 gyrNorm。

```
233 vqf_real_t angle = gyrNorm * f->gyrTs;
```
本步转过的角度 = 角速率 × 时间。用「轴角」方式积分比欧拉角积分精确得多。

```
234 if (gyrNorm > EPS) {
```
只有当角速度不为 0 时才需要更新四元数。

```
235 vqf_real_t c = cosf(angle/2);
```
半角余弦 c = cos(angle/2)。四元数表示旋转要用半角。

```
236 vqf_real_t s = sinf(angle/2)/gyrNorm;
```
每单位角速度对应的半角正弦：s = sin(angle/2)/gyrNorm。这样 s·gyrNoBias 就得到四元数的虚部。

```
237 vqf_real_t gyrStepQuat[4] = {c, s*gyrNoBias[0], s*gyrNoBias[1], s*gyrNoBias[2]};
```
构造本步的增量四元数 [c, s·ωx, s·ωy, s·ωz]。小角度下它近似为 [1, ω·dt/2]。

```
238 vqf_quatMultiply(f->gyrQuat, gyrStepQuat, f->gyrQuat);
```
左乘增量四元数，把旋转累加进 gyrQuat。这里输入输出是同一个数组，靠 vqf_quatMultiply 内部的临时变量保证正确。

```
239 vqf_normalize(f->gyrQuat, 4);
```
重新归一化。四元数连续相乘会积累舍入误差导致模长偏离 1，不归一化的话旋转矩阵会慢慢变形。

```
240 }
```
if 结束。

```
241
242 /* ----------------- 更新加速度计 ----------------- */
```
加速度处理区注释。

```
243 if (acc[0] == 0.0f && acc[1] == 0.0f && acc[2] == 0.0f) return;
```
全零加速度直接返回。这对应「传感器没数据」的情形，此时无法做重力观测，但陀螺部分的积分已经完成，所以是安全的提前退出。

```
244
245 if (f->restBiasEstEnabled) {
```
同样只在开启静止估计时做。

```
246 filterVec(acc, 3, f->restFilterTau, f->accTs, f->restAccLpB, f->restAccLpA,
```
对加速度做低通，输出到 restLastAccLp。

```
247 f->restAccLpState, f->restLastAccLp);
```
（续行）状态与输出。

```
248 f->restLastSquaredDeviations[1] = vqf_square(acc[0] - f->restLastAccLp[0])
```
算加速度波动量：原始与低通之差的平方和。匀速直线运动时加速度模值仍是 g
且方向不变，低通能跟上，所以波动小；一旦有真实机动，波动立刻变大。

```
249 + vqf_square(acc[1] - f->restLastAccLp[1]) + vqf_square(acc[2] - f->restLastAccLp[2]);
```
（续行）三轴平方和。

```
250
251 if (f->restLastSquaredDeviations[1] >= vqf_square(f->restThAcc)) {
```
加速度波动超阈值 → 不静止，计时清零。

```
252 f->restT = 0.0;
```
计时清零。

```
253 f->restDetected = false;
```
静止标志清除。

```
254 } else {
```
否则（加速度也平稳）：

```
255 f->restT += f->accTs;
```
疑似静止计时累加一个采样周期。

```
256 if (f->restT >= f->restMinT) f->restDetected = true;
```
累计够 restMinT（1.5 s）才真正判定静止。这一步是防止短暂的停顿被误认为静止。

```
257 }
```
if 结束。

```
258 }
```
条件块结束。

```
259
260 vqf_real_t accEarth[3];
```
声明地理系加速度，下面要存放变换后的加速度。

```
261 vqf_quatRotate(f->gyrQuat, acc, accEarth);
```
用陀螺四元数把机体加速度转到「地理系」。此时这个向量理论上应该指向竖直方向（重力反方向）。由于 gyrQuat 有漂移，实际会有偏差。

```
262 filterVec(accEarth, 3, f->tauAcc, f->accTs, f->accLpB, f->accLpA, f->accLpState,
f->lastAccLp);
```
对地理系加速度做低通滤波，结果存进 lastAccLp。这一层滤波的作用是**压制线性加速度和振动**——短时的运动加速度会被 3 秒时间常数抹平，只留下稳定的重力分量。这是 VQF 抗线性加速度的核心手段。

```
263
264 vqf_quatRotate(f->accQuat, f->lastAccLp, accEarth);
```
再用 accQuat 修正一次：把低通后的加速度从「包含历史修正」的坐标系转到「当前估计的重力方向」。这一步得到的是「用当前全部信息估计出的重力向量」。

```
265 vqf_normalize(accEarth, 3);
```
归一化，因为后面要反算倾角，必须是单位向量。

```
266
267 vqf_real_t accCorrQuat[4];
```
声明「重力校正四元数」，即本步要施加的小旋转。

```
268 vqf_real_t q_w = sqrtf((accEarth[2]+1)/2);
```
半角余弦 q_w = √((accEarth[2]+1)/2)。推导：把向量 v 转到 +z 的最短旋转四元数为 [1+v_z, v_y, −v_x, 0] 归一化；其模长为 √(2(1+v_z))，故实部 = (1+v_z)/√(2(1+v_z)) = √((1+v_z)/2)。

```
269 if (q_w > 1e-6f) {
```
当 q_w 足够大（即 v 不接近 −z）时走正常分支。1e-6 的阈值是为了避开 v 恰好指向 −z 时的奇异。

```
270 accCorrQuat[0] = q_w; accCorrQuat[1] = 0.5f*accEarth[1]/q_w;
```
实部 = q_w；虚部 x = 0.5·v_y / q_w，对应 (v × ẑ) 的 x 分量。

```
271 accCorrQuat[2] = -0.5f*accEarth[0]/q_w; accCorrQuat[3] = 0;
```
虚部 y = −0.5·v_x / q_w，对应 (v × ẑ) 的 y 分量；**虚部 z = 0**——这是 VQF 最精妙的一处：转轴落在水平面内，意味着这个校正**只改变倾斜、完全不改变偏航**，加速度计因此永远不会污染航向角。

```
272 } else {
```
else 分支：v 几乎指向 −z（姿态翻转了约 180°）。

```
273 accCorrQuat[0] = 0; accCorrQuat[1] = 1; accCorrQuat[2] = 0; accCorrQuat[3] = 0;
```
此时上面的公式退化，直接用一个绕 x 轴 180° 的旋转兜底。这种姿态在实际使用中极少出现。

```
274 }
```
if 结束。

```
275 vqf_quatMultiply(accCorrQuat, f->accQuat, f->accQuat);
```
左乘校正四元数，把倾斜修正累加进 accQuat。注意这里是**逐步累积**而非一次到位，所以 accQuat 平滑地跟随重力方向。

```
276 vqf_normalize(f->accQuat, 4);
```
归一化 accQuat，同样防止模长漂移。

```
277
278 f->lastAccCorrAngularRate = acosf(accEarth[2])/f->accTs;
```
记录本步校正的等效角速率：acos(重力估计的 z 分量)/Ts。重力估计越偏离竖直，acos 越大，表示「姿态错得越多」。这个量马上要作为卡尔曼的观测输入。

```
279
280 // 卡尔曼零偏估计 (Kalman Filter Bias Estimation)
```
卡尔曼零偏估计区的注释。

```
281 if (f->motionBiasEstEnabled || f->restBiasEstEnabled) {
```
只要两种估计方式有一个开启，就执行零偏更新。

```
282 vqf_real_t biasClip = f->biasClip*(M_PIf/180.0f);
```
把限幅值换成弧度/秒。

```
283 vqf_real_t accGyrQuat[4], R[9], biasLp[2];
```
声明三个局部量：合成四元数、旋转矩阵 R、投影后的零偏 biasLp。

```
284
285 vqf_quatMultiply(f->accQuat, f->gyrQuat, accGyrQuat);
```
合成四元数 accGyrQuat = accQuat ⊗ gyrQuat，其实就是最终姿态 q。

```
286 R[0] = 1 - 2*vqf_square(accGyrQuat[2]) - 2*vqf_square(accGyrQuat[3]);
```
由四元数构造旋转矩阵 R（9 个元素按行主序）。R[0] = 1 − 2(y² + z²) 是标准的 R00。

```
287 R[1] = 2*(accGyrQuat[2]*accGyrQuat[1] - accGyrQuat[0]*accGyrQuat[3]);
```
R[1] = 2(xy − wz)。

```
288 R[2] = 2*(accGyrQuat[0]*accGyrQuat[2] + accGyrQuat[3]*accGyrQuat[1]);
```
R[2] = 2(xz + wy)。

```
289 R[3] = 2*(accGyrQuat[0]*accGyrQuat[3] + accGyrQuat[2]*accGyrQuat[1]);
```
R[3] = 2(xy + wz)。

```
290 R[4] = 1 - 2*vqf_square(accGyrQuat[1]) - 2*vqf_square(accGyrQuat[3]);
```
R[4] = 1 − 2(x² + z²)。

```
291 R[5] = 2*(accGyrQuat[2]*accGyrQuat[3] - accGyrQuat[1]*accGyrQuat[0]);
```
R[5] = 2(yz − wx)。

```
292 R[6] = 2*(accGyrQuat[3]*accGyrQuat[1] - accGyrQuat[0]*accGyrQuat[2]);
```
R[6] = 2(xz − wy)。

```
293 R[7] = 2*(accGyrQuat[0]*accGyrQuat[1] + accGyrQuat[3]*accGyrQuat[2]);
```
R[7] = 2(yz + wx)。

```
294 R[8] = 1 - 2*vqf_square(accGyrQuat[1]) - 2*vqf_square(accGyrQuat[2]);
```
R[8] = 1 − 2(x² + y²)。至此 R 构造完毕。

```
295
296 biasLp[0] = R[0]*f->bias[0] + R[1]*f->bias[1] + R[2]*f->bias[2];
```
把零偏投影到地理系的前两个分量（水平方向）：biasLp = R 前两行 × bias。竖直分量不在这里处理，后面单独对待。

```
297 biasLp[1] = R[3]*f->bias[0] + R[4]*f->bias[1] + R[5]*f->bias[2];
```
（续行）第二行。

```
298
299 filterVec(R, 9, f->tauAcc, f->accTs, f->accLpB, f->accLpA, f->motionBiasEstRLpState, R);
```
对 R 矩阵的 9 个元素做低通，结果原地写回 R。滤波后 R
不再严格正交，但只用于权重计算，可以接受。目的是让卡尔曼的观测模型平滑，避免姿态抖动直接灌进零偏估计。

```
300 filterVec(biasLp, 2, f->tauAcc, f->accTs, f->accLpB, f->accLpA, f->motionBiasEstBiasLpState,
biasLp);
```
对 biasLp 的 2 个分量做低通。注意只有 2 个通道——竖直方向被刻意排除，因为它不可观测。

```
301
302 vqf_real_t w[3], e[3];
```
声明权重 w[3] 与新息 e[3]（观测残差）。

```
303 if (f->restDetected && f->restBiasEstEnabled) {
```
情况一：检测到静止，且静止估计已开启。

```
304 e[0] = f->restLastGyrLp[0] - f->bias[0]; e[1] = f->restLastGyrLp[1] - f->bias[1]; e[2] =
f->restLastGyrLp[2] - f->bias[2];
```
新息 = 平滑后的陀螺 − 当前零偏。物理含义：静止时真实角速度应为
0，所以「平滑陀螺读数」本身就是零偏的观测量。这是精度最高的一路观测。

```
305 matrix3SetToScaledIdentity(1.0, R);
```
观测矩阵退化为单位阵（下面几行会设置），这里先把 R 重置。

```
306 w[0] = w[1] = w[2] = f->biasRestW;
```
三个轴同权重，取静止权重 biasRestW。

```
307 } else if (f->motionBiasEstEnabled) {
```
情况二：运动中，且运动估计已开启。

```
308 e[0] = -accEarth[1]/f->accTs + biasLp[0] - R[0]*f->bias[0] - R[1]*f->bias[1] -
R[2]*f->bias[2];
```
水平第一轴新息：由「重力估计的倾斜速率」（−g_y/Ts）加上「投影后的零偏」再减去「当前零偏的预测」构成。姿态与重力方向不一致时，这个差就反映了水平零偏的大小。

```
309 e[1] = accEarth[0]/f->accTs + biasLp[1] - R[3]*f->bias[0] - R[4]*f->bias[1] - R[5]*f->bias[2];
```
（续行）第二轴：+g_x/Ts + …。

```
310 e[2] = - R[6]*f->bias[0] - R[7]*f->bias[1] - R[8]*f->bias[2];
```
竖直方向新息：只保留零偏自身的预测项。因为绕重力轴的零偏**不可观测**，没有可靠的观测量，所以这里不引入重力信息，全靠权重把它压住。

```
311 w[0] = f->biasMotionW; w[1] = f->biasMotionW; w[2] = f->biasVerticalW;
```
权重：水平两轴用 biasMotionW，竖直轴用被放大的 biasVerticalW。

```
312 } else {
```
else 分支：两种估计都没开。

```
313 w[0] = w[1] = w[2] = -1.0f;
```
权重置为 −1，作为「本步不做更新」的标记。

```
314 }
```
if 结束。

```
315
316 if (f->biasP[0] < f->biasP0) f->biasP[0] += f->biasV;
```
协方差预测（时间更新）：第一个对角线元素加过程噪声 V。完整形式应为 P += Q，VQF 采用对角近似以省算力。

```
317 if (f->biasP[4] < f->biasP0) f->biasP[4] += f->biasV;
```
第二个对角线。

```
318 if (f->biasP[8] < f->biasP0) f->biasP[8] += f->biasV;
```
第三个对角线，并给出「不超过 P0」的上限，防止协方差无限增长后失去响应。

```
319
320 if (w[0] >= 0) {
```
仅当权重有效（≥0）时执行测量更新。

```
321 vqf_clip(e, 3, -biasClip, biasClip);
```
新息限幅，防止异常观测把零偏一把拉飞。

```
322 vqf_real_t K[9];
```
声明卡尔曼增益矩阵 K。

```
323 matrix3MultiplyTpsSecond(f->biasP, R, K);
```
K = P•R^{T}（先算分子部分）。

```
324 matrix3Multiply(R, K, K);
```
K = R•K = R•P•R^{T}。

```
325 K[0] += w[0]; K[4] += w[1]; K[8] += w[2];
```
加上观测噪声权重：K = R·P·R^{T} + W（三个对角线分别加 w0/w1/w2）。

```
326 matrix3Inv(K, K);
```
求逆得到 (R·P·R^{T} + W)$^{-1}$，结果仍在 K 里。

```
327 matrix3MultiplyTpsFirst(R, K, K);
```
K = R^{T}**•K**。

```
328 matrix3Multiply(f->biasP, K, K);
```
K = P•K，最终得到卡尔曼增益 K = P·R^{T}·(R·P·R^{T} + W)$^{-1}$，正是标准形式。

```
329
330 f->bias[0] += K[0]*e[0] + K[1]*e[1] + K[2]*e[2];
```
更新零偏状态：bias += K·e，第一轴。

```
331 f->bias[1] += K[3]*e[0] + K[4]*e[1] + K[5]*e[2];
```
（续行）第二轴。

```
332 f->bias[2] += K[6]*e[0] + K[7]*e[1] + K[8]*e[2];
```
（续行）第三轴。

```
333
334 matrix3Multiply(K, R, K);
```
更新协方差：先算 K·R。

```
335 matrix3Multiply(K, f->biasP, K);
```
再算 K = K·R·P。

```
336 for(size_t i = 0; i < 9; i++) f->biasP[i] -= K[i];
```
P −= K•R•P，即 P ← (I − KR)P，标准卡尔曼协方差更新。

```
337
338 vqf_clip(f->bias, 3, -biasClip, biasClip);
```
零偏限幅：把估计值钳在 ±biasClip 内，防止病态输入导致发散。

```
339 }
```
if 结束（权重有效）。

```
340 }
```
if 结束（运动/静止估计总开关）。

```
341 }
```
vqf_update 函数结束。

```
342
343 void vqf_output(struct VQF_FILTER_t *f)
```
输出函数：把内部状态换算成对外的姿态量。

```
344 {
```
函数体开始。

```
345 vqf_quatMultiply(f->accQuat, f->gyrQuat, f->q);
```
合成最终四元数 q = accQuat ⊗ gyrQuat。顺序不能反：先应用陀螺的旋转，再用加速度校正，几何上相当于「陀螺提供快速姿态，加速度提供慢速基准」。

```
346
347 float q0 = f->q[0], q1 = f->q[1], q2 = f->q[2], q3 = f->q[3];
```
把四个分量取到局部变量，减少结构体解引用。

```
348
349 float sinp = 2.0f * (q0 * q2 - q1 * q3);
```
先算 sinp = 2(q0·q2 − q1·q3)，用于俯仰角。

```
350 if (sinp > 1.0f) sinp = 1.0f;
```
上限保护：钳到 1.0。

```
351 if (sinp < -1.0f) sinp = -1.0f;
```
下限保护：钳到 −1.0。浮点误差可能让 sinp 变成 1.0000001，asinf 会返回 NaN
并污染全部输出，因此这一步限幅是必须的。

```
352 f->pitch = asinf(sinp) * RAD2DEG;
```
俯仰角 pitch = asin(sinp) 并转成角度。ZYX 欧拉序下 pitch 是绕 y 轴的角，范围本就是 ±90°，用 asin 而非 atan2。

```
353 f->roll = atan2f(2.0f * (q0 * q1 + q2 * q3), 1.0f - 2.0f * (q1 * q1 + q2 * q2)) * RAD2DEG;
```
横滚角 roll = atan2(2(q0q1 + q2q3), 1 − 2(q1² + q2²))，最后转成角度。

```
354 f->yaw = atan2f(2.0f * (q1 * q2 + q0 * q3), 1.0f - 2.0f * (q2 * q2 + q3 * q3)) * RAD2DEG;
```
偏航角 yaw = atan2(2(q1q2 + q0q3), 1 − 2(q2² + q3²))。范围自然是 ±180°。

```
355
356 float yaw_diff = f->yaw - f->last_yaw;
```
算与上一帧的偏航差，用于检测跨越 ±180°。

```
357 if (yaw_diff > 180.0f) {
```
若差值大于 180°，说明刚才是从 +180° 跳到了 −180° 附近，跨过了边界。

```
358 yaw_diff -= 360.0f;
```
把差值修正回真实的小角度。

```
359 f->yaw_laps --;
```
圈数 −1。注意方向：yaw 减小时差值会表现为 +360，故这里减 1。

```
360 }
```
if 结束。

```
361 else if (yaw_diff < -180.0f) {
```
else if：差值小于 −180°。

```
362 yaw_diff += 360.0f;
```
同样修正回真实角度。

```
363 f->yaw_laps ++;
```
圈数 +1。

```
364 }
```
if/else 结束。

```
365 f->YawTotalAngle += yaw_diff;
```
累加进累计偏航角。由于 yaw_diff 已经解开，YawTotalAngle 可以连续增长超过 ±360°。

```
366 f->last_yaw = f->yaw;
```
记录本次 yaw，供下一帧算差值。

```
367 }
```
vqf_output 函数结束。

```
368
```

八、VQF 与 EKF 的对比

8.1 先明确「EKF」指什么

EKF（Extended Kalman Filter，扩展卡尔曼滤波）是一套**方法论**，而不是一个具体算法。用于姿态估计时，典型配置是：

- 状态量：姿态（四元数 q 或旋转矩阵 R）+ 陀螺零偏 b，有时还包含速度、位置（组合导航时）。
- 预测：用陀螺积分推进姿态，同时把协方差按 F = ∂f/∂x 传播。
- 更新：用加速度计（和磁力计/GNSS）构造观测方程，按 H = ∂h/∂x 线性化，执行标准卡尔曼更新。
- 调参：过程噪声 Q 和观测噪声 R 矩阵。

所以下面的比较，是**VQF 这个具体算法**与**姿态估计型 EKF 这一方法**的对比。

8.2 核心差异一览

维度	VQF	姿态 EKF
核心机制	确定性几何构造 + 分层滤波	概率状态估计 + 雅可比线性化
姿态的状态表示	两个独立四元数：gyrQuat 与 accQuat	单个姿态量（q 或 R）及其误差状态
是否需要雅可比	**不需要**，全程没有线性化	**需要**，每步计算 F、H 矩阵
协方差传播	仅对 3 维零偏做 3×3 协方差	对完整状态做（姿态 3~4 维 + 零偏 3 维 + 可选更多）
计算量	低：四元数乘法 + 几个 3×3 矩阵	中到高：矩阵乘法、求逆、雅可比，状态维数越高越重
可观测性处理	显式：给竖直方向一个大权重，直接压死	隐式：体现在协方差与 H 矩阵，需仔细设计 Q/R
调参对象	物理量：时间常数 tau、静止阈值、零偏 σ	统计量：Q、R 矩阵（物理意义较间接）
加速度计影响范围	**严格只影响倾斜**（数学上保证 z 分量为 0）	取决于 H 矩阵设计，设计不严会漏进偏航
抗线性加速度	靠地理系低通（tau = 3 s）硬压	靠调大 R 或做加速度有效性检测
磁力计支持	原版支持，本实现未使用	常规支持，观测方程中增加一项
数值稳定性	归一化 + 限幅 + NaN 哨兵，鲁棒	协方差可能失去正定性/对称性，需额外处理
实现难度	中等，每步计算路径确定	较高，雅可比容易写错且难调试
典型应用	可穿戴、机器人、无人机姿态（6 轴为主）	组合导航、GNSS/INS 融合、高精度定位

8.3 三个最本质的区别

区别一：概率框架 vs 几何框架

EKF 把姿态看作一个**概率分布**。它维护均值（姿态）和协方差（不确定度），每次观测更新都是「按各自的不确定度加权平均」——这是统计意义上的最优估计（在高斯假设下）。

VQF 不做概率推断。它用**确定性的几何运算**定义「应该怎么修正」：算出把重力向量旋正所需的最短旋转，然后按固定比例施加。「不确定度」的角色由低通时间常数和阈值这类**确定性参数**扮演。

这带来一个实际差别：**VQF 的行为更容易预测和调参。**调大 tau 就是更平滑，因果直接。而在 EKF 里调大 R 的影响会通过协方差耦合到其他状态，有时会出现「调了 A 结果 B 变差」的非直观现象。

区别二：偏航通道的处理

这是工程上最有价值的一条。

VQF：因为校正四元数的 z 分量**恒为 0**，加速度计在数学上就**无法影响偏航。**这是构造性的保证，不依赖调参。

EKF：偏航的不可观测性是靠 H 矩阵和协方差隐式表达的。如果 H 的推导有细微错误，或者 Q/R 设置不当，加速度计的观测噪声就会渗入偏航通道，表现为「静止时航向缓慢乱转」。这类问题在实际调试中相当常见，而且从输出上很难定位到根因。

区别三：计算量与实时性

VQF 每步的计算量基本是固定的常数：几次四元数乘法、三次向量低通（每次 3 通道 × 2 阶）、一个 3×3 求逆。没有任何随状态维数增长的运算。

EKF 的计算量随状态维数**立方增长**（主要是矩阵求逆）。姿态估计型 EKF 通常是 7 维状态（4 元数 + 3 零偏）或 6 维误差状态，在 Cortex-M4F 上可以跑，但留给其他任务的时间会明显减少。如果扩展到 15 维（含速度位置），MCU 上的实时性就要认真评估了。

8.4 各自更适合什么场景

场景	建议	理由
6 轴 IMU 姿态估计（无磁力计）	**VQF**	专为此设计；偏航不被污染；算力占用低
嵌入式 MCU、多任务实时系统	**VQF**	计算量常数级，便于做任务周期规划
需要复杂调参、追求统计最优	EKF	有成熟的协方差调优方法论
融合磁力计 / GNSS / 轮速里程计	**EKF**	多源异构观测的统一框架，VQF 不擅长
需要速度、位置等衍生状态	**EKF**	VQF 只输出姿态
对「偏航零漂」极度敏感	**VQF**	构造性保证偏航不被重力观测污染
需要严格的不确定度输出	EKF	VQF 不提供协方差

8.5 一句话总结

EKF 是「用统计学统一处理一切观测」的通用框架，VQF 是「用几何学精确处理重力这一种观测」的专用工具。

在 6 轴姿态估计这个具体问题上，VQF 用更少的计算量和更易理解的物理参数，换取了与 EKF 相当甚至更好的鲁棒性——尤其体现在偏航通道不受重力观测污染这一点上。但一旦需要融合磁力计、GNSS 或估计速度位置，EKF 的通用性就不可替代。

九、本工程中的集成方式

本工程的调用点在 App/Src/INS.c，周期为 4 ms（250 Hz）：

```
// 初始化：dt = 0.004 s
vqf_init(&vqf_filter, 0.004f);

// 每步更新
vqf_update(&vqf_filter, gyro[0], gyro[1], gyro[2],
accel[0], accel[1], accel[2]);
vqf_output(&vqf_filter);

// 取用结果
data_ins.pitch = vqf_filter.pitch;
data_ins.roll = vqf_filter.roll;
data_ins.yaw = -vqf_filter.yaw; // 注意取负，坐标系约定
data_ins.q[0..3] = vqf_filter.q[0..3];
data_ins.yaw_total = vqf_filter.YawTotalAngle;
```

9.1 三个需要留意的点

① dt 必须与真实调用周期一致

代码 166~167 行把 dt 存进 gyrTs/accTs，它同时决定了：

- 四元数积分角度 θ = |ω|·dt —— dt 偏大则姿态转得比实际快。
- 滤波器系数的预畸变 C = tan(π·fc·Ts) —— dt 偏大则滤波带宽偏离设计值。
- 静止检测计时 restT += accTs —— dt 偏大则静止判定过慢。
- lastAccCorrAngularRate = acos(...)/accTs —— 直接影响零偏估计的量纲。

所以 vqf_init(&vqf;_filter, 0.004f) 里的 0.004 **必须与 INS_Task 的实际周期严格一致。**如果 INS_Task 的调度周期抖动或被高优先级任务抢占，这几处都会同时产生偏差。

② 欧拉角 vs 四元数

本工程两者都取用了。建议：

- **显示/调试**用 pitch/roll/yaw，直观。
- **串级 PID 的外环**用四元数或 YawTotalAngle，避免万向锁和 ±180° 跳变。
- 特别提醒：**不要把 pitch/roll 直接送进三环 PID**，在 pitch 接近 ±90° 时欧拉角会退化，导致控制发散。

③ 加速度计数据的有效性

代码 243 行有 if (acc[0]==0 && acc[1]==0 && acc[2]==0)
return;，这是针对「传感器掉线」的保护。但要注意：它**只检测全零**，如果 BMI088 返回的是固定垃圾值而非零，这个保护不会触发，错误的加速度会污染姿态。生产环境建议在上层再做一次合理性检查（如模长是否接近 9.8 m/s²）。

十、参数速查表

参数	默认值	作用	调大后
tauAcc	3.0 s	加速度低通时间常数	更平滑、更抗机动，但收敛慢
biasSigmaInit	0.5 °/s	零偏初始不确定度	初期收敛更快、更抖
biasForgettingTime	100 s	零偏遗忘时间	零偏更稳但适应慢
biasClip	2.0 °/s	零偏限幅	允许更大零偏，但风险高
biasSigmaMotion	0.1 °/s	运动过程噪声	运动时收敛快、更抖
biasVerticalForgettingFactor	0.0001	竖直方向遗忘因子	越大→竖直方向更新越多，**不建议修改**
biasSigmaRest	0.03 °/s	静止过程噪声	静止估计更激进
restMinT	1.5 s	静止判定最短时长	更保守，不易误判
restFilterTau	0.5 s	静止检测低通	抗噪更好但判定迟钝
restThGyr	2.0 °/s	静止陀螺阈值	更容易判为静止
restThAcc	0.5 m/s²	静止加速度阈值	更容易判为静止

10.1 调参优先顺序建议

- **先确认 dt 正确。**参数再准，采样周期错了全盘皆错。
- 再调 tauAcc：振动大就先增大；响应跟不上就减小。
- 然后确认静止检测能正常工作（静止时 restDetected 应为 true）。
- 最后才动零偏参数。**biasVerticalForgettingFactor 一般不要改**——它关系到偏航零偏的稳定性。

附录：关于本文档

本文档中引用的所有代码，均直接从以下文件读取并逐行呈现，未做任何改写，以保证与工程源码完全一致：

- Library/Src/VQF_filter.c（367 行）
- Library/Inc/VQF_filter.h（86 行）

代码字体使用 Maple Mono CF CN——与 VSCode 当前设置一致。正文使用微软雅黑。

生成脚本位于 docs/vqf_pdf.py 与 docs/vqf_content.py，修改源码或解释文字后重新运行 python docs/vqf_pdf.py 即可重新生成。